MINISTRY OF EDUCATION AND TRAINING

FPT UNIVERSITY

AI-driven Software System for Recruitment Screening using NLP and Semantic Information Retrieval

A Ranking-based Approach for English IT CV–JD Matching

by

Tran Viet Hung

A thesis submitted in conformity with the requirements

for the degree of Master of Software Engineering

© Copyright by Tran Viet Hung 2026

MINISTRY OF EDUCATION AND TRAINING

FPT UNIVERSITY

AI-driven Software System for Recruitment Screening using NLP and Semantic Information Retrieval

A Ranking-based Approach for English IT CV–JD Matching

by

Tran Viet Hung

A thesis submitted in conformity with the requirements

for the degree of Master of Software Engineering

Supervisors:

Dr. Doan Xuan Huy Minh

© Copyright by Tran Viet Hung 2026

Abstract

Recruitment screening is a high-volume information filtering problem in which recruiters must identify a small number of relevant candidates from a large collection of curriculum vitae (CVs). Existing applicant tracking systems and similarity-based methods often rely heavily on exact keywords or overall semantic similarity. Consequently, they may rank a CV highly even when the candidate does not satisfy mandatory experience, seniority, responsibility, or evidence requirements. A fresher CV, for example, may mention Java, Spring Boot, SQL, and Docker and therefore appear relevant to a senior Java role despite lacking the required years of experience, production ownership, system design, and mentoring evidence.

This thesis proposes an AI-driven recruitment screening system that formulates English IT CV–JD matching as a ranking problem and introduces a *requirement-aware and evidence-aware matching mechanism with constraint-based reranking*. The system ingests PDF CVs, applies direct text extraction, and falls back to optical character recognition (OCR) for scanned documents. It then parses CV and job description (JD) content into structured sections, decomposes each JD into mandatory and optional requirements, retrieves supporting evidence from Work Experience and Projects, classifies each requirement as satisfied, partially satisfied, unsupported, or not satisfied, and validates hard constraints such as minimum years of experience and seniority level.

The experimental design combines `Pending` real scanned PDF CVs from a public Kaggle dataset with approximately 1,000 controlled AI-generated English IT CVs. The real corpus is used for OCR, parsing, and practical ranking evaluation, whereas the synthetic corpus supports controlled seniority-mismatch, unsupported-claim, keyword-stuffing, and scalability experiments. Synthetic data is not used as the sole evidence of matching effectiveness; the primary ranking conclusions are based on a manually judged CV–JD evaluation pool.

The proposed approach is compared with TF–IDF, SBERT full-document similarity, and section-level SBERT baselines. Evaluation uses NDCG@5, NDCG@10, MAP, MRR, Recall@10, Recall@20, hard-constraint violation rate, unsupported-claim rate, OCR reliability, operational latency, and a small explanation-usability assessment. Quantitative values

remain `Pending final experiment` until the labeled evaluation set and reproducible runs are completed. The intended outcome is a practical and auditable decision-support prototype that improves initial CV screening while keeping final hiring decisions under human control.

Keywords: recruitment screening, CV–JD matching, semantic information retrieval, requirement decomposition, evidence-aware matching, constraint-based reranking, SBERT, OCR, explainable AI.

Acknowledgments

I would like to express my sincere gratitude to my supervisor, Dr. Doan Xuan Huy Minh, for his valuable guidance, constructive feedback, and continued support throughout the development of this thesis. His comments helped me refine the research problem, improve the academic structure, and move the study beyond a simple similarity-scoring approach toward a more practical requirement-aware and evidence-aware CV–JD matching mechanism. His suggestions were especially important in clarifying the research gap, strengthening the evaluation design, and ensuring that the proposed solution remained aligned with real recruitment challenges.

I am also grateful to the lecturers of the Master of Software Engineering program at FPT University for providing the knowledge and academic foundation that supported this research. The courses in software engineering, system design, data processing, artificial intelligence, machine learning, and research methodology contributed significantly to my ability to design, implement, and evaluate the proposed system.

I would like to thank my colleagues and professional contacts who shared practical insights into recruitment screening, software development, project delivery, and technical candidate evaluation. Their perspectives helped me better understand the difficulties recruiters and technical reviewers face when assessing large numbers of CVs, especially when candidate information is incomplete, inconsistently formatted, or expressed using different technical terminology. These observations influenced the design of the requirement decomposition, evidence verification, seniority validation, and human-in-the-loop review mechanisms presented in this thesis.

I also appreciate the contributions of the reviewers and individuals who supported the evaluation preparation, including the development of representative job descriptions, relevance-labeling criteria, and feedback on candidate-ranking outputs. Their input helped make the evaluation more realistic and ensured that the proposed system was assessed not only through technical metrics but also through practical recruitment considerations.

My sincere thanks also go to the researchers, open-source communities, and dataset providers whose work made this study possible. Publicly available resume datasets, natural language processing libraries, sentence-transformer models, OCR tools, and software

frameworks provided an essential foundation for implementing and evaluating the prototype in a reproducible manner.

Finally, I am deeply grateful to my family for their patience, understanding, encouragement, and emotional support throughout the research and writing process. Balancing professional responsibilities, academic work, system implementation, and experimental preparation required significant time and effort. Their continuous motivation gave me the strength to overcome difficulties and complete this thesis.

This work would not have been possible without the guidance, knowledge, support, and encouragement of all those mentioned above.

Contents

Abstract **1**

Acknowledgments **3**

1 Introduction **11**

1.1 Background 11

1.2 Problem Statement 11

1.3 Research Aim and Objectives 12

1.4 Research Questions 13

1.5 Scope and Exclusions 14

1.6 Main Contributions 14

1.7 Thesis Structure 15

2 Literature Review **16**

2.1 Recruitment Screening and Applicant Tracking Systems 16

2.2 Information Retrieval for CV–JD Matching 16

2.3 Semantic Retrieval and SBERT 16

2.4 Cross-Encoder and Reranking Approaches 17

2.5 Requirement Decomposition and Constraint Matching 17

2.6 Evidence-aware Matching and Unsupported Claims 17

2.7 Resume Parsing, PDF Extraction, and OCR 18

2.8 Explainability and Human Control 18

2.9 Comparison of Related Approaches 18

2.10 Synthetic Data for Controlled Evaluation 18

2.11 Research Gap 20

3 System Design and Implementation **21**

3.1 System Overview . . . 21

3.2 Functional Requirements . . . 21

3.2.1 Non-functional Requirements . . . 22

3.3 PDF Ingestion and OCR Fallback . . . 22

3.4 Structured CV and JD Parsing . . . 22

3.5 Dataset Provenance and Source Tracking . . . 24

3.6 Synthetic CV Generation Protocol . . . 25

3.7 Requirement Decomposition . . . 26

3.8 Candidate Evidence Retrieval . . . 26

3.9 Requirement Satisfaction Assessment . . . 26

3.9.1 Operational Decision Rules . . . 27

3.10 Experience and Seniority Validation . . . 28

3.11 Constraint-based Reranking . . . 29

3.11.1 Reranking Procedure . . . 29

3.12 Baseline Methods . . . 30

3.13 Explanation Generation . . . 30

3.14 Software Architecture . . . 32

3.15 API Design . . . 32

3.16 Security and Human-in-the-loop Controls . . . 32

4 Evaluation and Discussion **34**

4.1 Evaluation Goals . . . 34

4.2 Dataset and Test Corpus . . . 34

4.2.1 Real Scanned PDF Corpus . . . 34

4.2.2 Controlled Synthetic CV Corpus . . . 36

4.2.3 Combined Retrieval Corpus . . . 36

4.2.4 Human-labeled Evaluation Pool . . . 36

4.2.5 Controlled Robustness Sets . . . 36

4.3 Relevance Labeling . . . 37

4.3.1 Reviewer Agreement and Explanation Usability . . . 39

4.4 Experimental Models . . . 40

4.5 Ranking Metrics . . . 40
4.6 Ablation Study . . . 41
4.6.1 Threshold Sensitivity . . . 41
4.7 Seniority Mismatch Robustness Test . . . 41
4.8 Keyword-stuffing Robustness Test . . . 41
4.9 PDF and OCR Evaluation . . . 42
4.10 Result Tables . . . 42
4.11 Case-study Analysis . . . 42
4.12 Operational Performance . . . 44
4.13 Error Analysis . . . 44
4.14 Discussion . . . 44
4.15 Threats to Validity . . . 45
4.15.1 Internal Validity . . . 45
4.15.2 Construct Validity . . . 46
4.15.3 External Validity . . . 46
4.15.4 Conclusion Validity . . . 46

5 Conclusion and Future Work **47**
5.1 Conclusion . . . 47
5.2 Main Contributions . . . 47
5.3 Limitations . . . 48
5.4 Future Work . . . 48

A Evaluation Artifacts **50**
A.1 Final Dataset Record . . . 50
A.2 Synthetic Generation Protocol . . . 50
A.3 Labeling Instructions . . . 50
A.4 Experiment Configuration . . . 51
A.5 Prototype Evidence . . . 51
A.6 Pending Completion Checklist . . . 51

References **52**

List of Figures

1.1 Recruitment matching problem and research objective 12

3.1 Proposed requirement-aware and evidence-aware matching architecture . 21
3.2 PDF ingestion and OCR fallback process 23
3.3 Structured parsing output for CV and JD documents 24
3.4 Human-in-the-loop recruiter review workflow 31

4.1 Evaluation workflow for effectiveness, robustness, and data quality 35

List of Tables

1.1 Research scope and exclusions 14

2.1 Comparison of related CV–JD matching approaches 19

3.1 Structured representation of a job description 24
3.2 Core provenance and synthetic manifest fields 25
3.3 Requirement satisfaction states 27
3.4 Operational requirement assessment rules 28
3.5 Definition of constraint-based reranking components 29
3.6 Implementation and experiment environment 32
3.7 Main software modules 33
3.8 Main API endpoints 33

4.1 Dataset composition for the final experiment 37
4.2 Dataset documentation fields for the final experiment 38
4.3 Manual relevance scale 38
4.4 Reviewer labeling rubric 39
4.5 Explanation usability reporting template 39
4.6 Experimental model comparison 40
4.7 Keyword-stuffing test design 42
4.8 Real-data ranking effectiveness results 43
4.9 Controlled robustness results 43
4.10 Real scanned PDF OCR and parsing results 43
4.11 Combined-corpus operational performance 43
4.12 Seniority mismatch case-study template 44
4.13 Keyword-stuffing case-study template 44

4.14 Error analysis categories 45

A.1 Items requiring external input or experiment execution 51

Chapter 1

Introduction

1.1 Background

Recruitment screening is one of the earliest and most time-consuming stages in the hiring process. For a single opening, recruiters may receive tens or hundreds of CVs. The challenge is particularly significant in information technology recruitment because technical roles are described using diverse terminology, abbreviations, frameworks, cloud platforms, and project responsibilities. A backend position may require Java, Spring Boot, REST APIs, microservices, SQL, Docker, Kubernetes, and cloud deployment, while candidates may describe similar experience using different phrases such as enterprise services, distributed systems, containerized deployment, or cloud-native development.

Traditional applicant tracking systems generally support keyword search, Boolean rules, and manual filters. These tools are efficient for exact matching but are sensitive to vocabulary mismatch and keyword stuffing. Semantic retrieval models reduce vocabulary mismatch by representing CVs and JDs as dense vectors. However, semantic similarity alone does not guarantee actual suitability. Two documents can be semantically related even when the candidate fails a critical requirement. A junior candidate may use the same technology stack as a senior candidate, but the depth, duration, ownership, and context of experience are different.

Therefore, the central problem is not only to calculate text similarity, but to determine whether the candidate satisfies the individual requirements of the JD and whether the CV contains supporting evidence for those requirements.

1.2 Problem Statement

Given a JD q and a CV collection $D = \{d_1, d_2, \ldots, d_n\}$, a conventional ranking system calculates a relevance score $s(q, d_i)$ and returns an ordered list. This formulation is useful,

but a single similarity score can hide important mismatches. For example:

- a candidate may match the required skills but not the minimum years of experience;
- a CV may list a technology in the Skills section without evidence in Work Experience or Projects;
- an intern or fresher may be ranked highly for a senior role because of lexical or semantic overlap;
- scanned PDF extraction errors may remove important evidence and produce incorrect rankings.

The problem addressed in this thesis is therefore:

> How can an AI-driven recruitment screening system rank English IT CVs for a JD by considering not only semantic similarity, but also requirement satisfaction, mandatory constraints, seniority alignment, and supporting evidence?

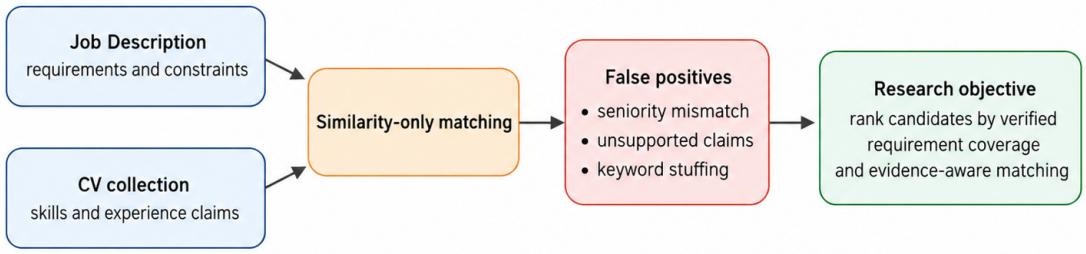


Figure 1.1: Recruitment matching problem and research objective

1.3 Research Aim and Objectives

The aim is to design and evaluate a practical recruitment screening system that performs requirement-aware and evidence-aware CV–JD matching.

The objectives are:

1. Formulate English IT CV–JD matching as a ranking and information retrieval problem.
2. Build a PDF-first CV ingestion pipeline with OCR fallback for scanned PDFs.
3. Parse CVs and JDs into structured sections and requirements.
4. Decompose JDs into mandatory and optional requirements.
5. Retrieve evidence from CV Work Experience and Projects for each requirement.
6. Validate hard constraints, including minimum experience and seniority level.
7. Rerank candidates using verified requirement coverage rather than overall similarity alone.
8. Evaluate the proposed mechanism using ranking metrics, robustness tests, and error analysis.
9. Provide recruiter-facing explanations while preserving human control over hiring decisions.

1.4 Research Questions

RQ1
How effective are TF–IDF, SBERT full-document similarity, and section-level SBERT as baseline methods for English IT CV–JD ranking?

RQ2
Does requirement-aware matching improve ranking quality compared with similarity-only ranking?

RQ3
Does evidence-aware verification reduce false positives caused by unsupported skill claims and keyword stuffing?

RQ4
Does constraint-based reranking reduce false positives caused by experience and seniority mismatch?

RQ5
How useful and understandable are the requirement-level explanations to recruiters or technical reviewers during candidate review?

1.5 Scope and Exclusions

The scope is intentionally limited to English IT recruitment and initial CV screening. The system prioritizes review order and provides evidence; it does not make final hiring or rejection decisions.

Table 1.1: Research scope and exclusions

Dimension	Included	Excluded or Future Work
Language	English CVs and JDs	Vietnamese and multilingual matching
Domain	IT roles such as developer, QA, DevOps, data, BA, PM, and architect	Non-IT recruitment
Input	PDF CVs with direct extraction and OCR fallback	Unsupported office and image formats as primary inputs
Dataset	Real scanned Kaggle PDFs and controlled AI-generated English IT CVs	Private company CV databases
Task	Top-k ranking and recruiter decision support	Automatic hiring or rejection
Matching	Requirement-aware, evidence-aware, constraint-based reranking	External employment or credential verification
Ground truth	Human relevance judgments and controlled robustness metadata	Fully automatic relevance labeling
Models	TF–IDF, SBERT, section-level SBERT, and the proposed mechanism	Large-scale model fine-tuning and production ATS integration
Evaluation	Ranking metrics, ablation, robustness, OCR, latency, and explanation usability	Long-term employee performance prediction

1.6 Main Contributions

The thesis contributes:

1. a modular software architecture for PDF ingestion, OCR fallback, structured parsing, requirement decomposition, evidence retrieval, constraint validation, reranking, and explanation;
2. a requirement-level representation that classifies each JD requirement as satisfied, partially satisfied, unsupported, or not satisfied;

3. an explicit distinction between declared skills and evidence-supported skills in Work Experience and Projects;
4. explicit treatment of minimum experience and seniority as constraints rather than generic semantic similarity;
5. a deterministic lexicographic reranking mechanism that prevents broad similarity from compensating for failed mandatory requirements;
6. a mixed real-and-synthetic experimental design that separates real-world OCR and human-labeled ranking evaluation from controlled robustness and scalability testing;
7. recruiter-oriented explanations that present requirement coverage, evidence sentences, gaps, and warnings;
8. an evaluation protocol covering effectiveness, ablation, seniority mismatch, keyword stuffing, OCR quality, operational latency, and explanation usability.

The novelty is not a new foundation language model. It is the design and validation of an auditable matching mechanism that changes the unit of comparison from whole-document similarity to individual JD requirements and their supporting CV evidence. This distinction links the algorithmic behavior, software architecture, recruiter workflow, and evaluation criteria to the original recruitment problem.

1.7 Thesis Structure

The remainder of the thesis is organized into four chapters. Chapter 2 reviews relevant literature. Chapter 3 presents the system design and implementation. Chapter 4 describes the evaluation and discusses the results. Chapter 5 concludes the thesis and proposes future work.

Chapter 2

Literature Review

2.1 Recruitment Screening and Applicant Tracking Systems

Applicant tracking systems support CV storage, filtering, keyword search, and workflow management. Keyword and Boolean filters are efficient and interpretable, but they are vulnerable to vocabulary mismatch and keyword stuffing. A candidate may be relevant without using the exact terms in the JD, while another candidate may list many keywords without demonstrating practical experience.

2.2 Information Retrieval for CV–JD Matching

In information retrieval, the JD is treated as a query and the CV collection as documents. TF–IDF remains a useful lexical baseline because exact technical terms often matter in IT recruitment. Cosine similarity between TF–IDF vectors is efficient and transparent. BM25 improves lexical retrieval through term saturation and document-length normalization, but it is not the main implementation in this thesis and remains a future extension [5, 8].

2.3 Semantic Retrieval and SBERT

Transformer-based embeddings capture semantic relationships beyond exact words. BERT introduced contextual bidirectional representations [2], while Sentence-BERT enables efficient sentence and document similarity by producing comparable embeddings [6]. SBERT is suitable for first-stage retrieval because CV and JD embeddings can be precomputed and compared efficiently.

However, full-document similarity has limitations. Long CVs may contain unrelated content, and a high semantic score can be produced by broad topical overlap. Section-level matching reduces this problem by comparing relevant sections, such as Skills and Work

Experience, separately.

2.4 Cross-Encoder and Reranking Approaches

Two-stage retrieval architectures commonly use a fast bi-encoder to retrieve a candidate set and a more expensive reranker to refine ordering. Cross-encoders jointly process query-document pairs and often provide stronger interaction modeling than independent embeddings. Learning-to-rank methods such as LambdaMART can combine multiple relevance signals, but they require sufficient labeled data. Given the time and data limitations of this thesis, the proposed system uses deterministic requirement-level reranking and places trainable learning-to-rank in future work.

2.5 Requirement Decomposition and Constraint Matching

A JD is not a single homogeneous text. It contains mandatory skills, optional skills, minimum experience, seniority, responsibilities, education requirements, and certifications. Treating these requirements independently is closer to recruiter practice. Hard constraints determine whether a candidate satisfies critical conditions, while soft constraints affect preference without necessarily excluding the candidate.

Constraint-aware matching is especially important for seniority. Semantic similarity between a senior JD and a junior CV may be high if both mention the same technologies. A dedicated seniority check can identify this mismatch directly.

2.6 Evidence-aware Matching and Unsupported Claims

A CV may declare skills without providing evidence. It is not possible to determine whether a CV is fraudulent using the document alone, but it is possible to identify *unsupported claims*. Evidence-aware matching searches Work Experience and Project sections for sentences that support a declared skill or responsibility. This distinction reduces ranking inflation caused by keyword stuffing.

The evidence problem can be represented as:

$$E(r, d) = \max_{s \in \mathcal{S}(d)} \text{sim}(r, s), \tag{2.1}$$

where r is a JD requirement, $\mathcal{S}(d)$ is the set of evidence sentences extracted from CV d, and sim is a semantic similarity function. If no sentence exceeds a threshold, the requirement is marked unsupported or not satisfied.

2.7 Resume Parsing, PDF Extraction, and OCR

CV documents often contain multiple columns, icons, tables, and irregular headings. Text-based PDFs can be parsed using libraries such as PyPDF or pdfplumber. Scanned PDFs require OCR, for which Tesseract provides an established open-source engine architecture [15]. OCR may introduce character errors, broken lines, and section boundary problems, which can affect ranking quality. Therefore, the ingestion pipeline should record extraction status, text length, and parsing warnings.

2.8 Explainability and Human Control

Recruitment systems require transparency because screening affects individuals. Practical explanations should show job-relevant evidence that recruiters can verify. Attention heatmaps alone are not sufficient because attention is not necessarily a faithful explanation of model behavior [4]. The proposed system therefore explains ranking through requirement status, missing constraints, seniority gaps, and supporting sentences.

2.9 Comparison of Related Approaches

Table 2.1 summarizes the main methodological families relevant to this thesis. Lexical and dense retrieval methods are effective for candidate discovery but do not directly verify requirement satisfaction. Cross-encoders and learning-to-rank methods improve interaction modeling, but they normally require additional computation or labeled training data. The proposed mechanism focuses on requirement decomposition, evidence verification, and explicit constraint handling while remaining deterministic and auditable.

2.10 Synthetic Data for Controlled Evaluation

Synthetic data can supplement limited real-world corpora by creating controlled combinations of role, seniority, experience, skill evidence, and adversarial cases. However, AI-generated text may contain stylistic artifacts, overly clean language, or patterns inherited from the generation prompt. Therefore, synthetic CVs should support robustness and scalability analysis rather than replace human judgment on real CVs. Recent work on LLM-driven synthetic data emphasizes the need for transparent generation protocols, quality controls, source separation, and careful interpretation of downstream results [13, 14].

In this thesis, synthetic CVs are generated from structured candidate profiles rather than copied from target JDs. Each record retains provenance and scenario metadata, allowing controlled tests for seniority mismatch, missing mandatory skills, unsupported claims, and

Table 2.1: Comparison of related CV–JD matching approaches

Approach	Main signal	Typical output	Strength	Limitation relative to this work
Keyword / TF–IDF	Exact term overlap	Similarity rank	Fast and interpretable	Vocabulary mismatch; vulnerable to keyword stuffing
Dense bi-encoder / SBERT	Global semantic similarity	Similarity rank	Handles paraphrases and related wording	Can hide mandatory, seniority, and evidence gaps
Section-level matching	Section-specific semantic similarity	Weighted rank	Reduces dilution from unrelated CV content	Usually remains score-centric and does not verify each requirement
Cross-encoder reranking	Joint CV–JD interaction	Relevance score	Strong pairwise interaction modeling	Higher latency; difficult to audit without evidence extraction
Learning-to-rank	Learned feature combination	Learned rank	Learns feature importance from labels	Requires sufficient representative labeled query groups
Proposed mechanism	Requirement states, evidence, constraints	Auditable rank	Prioritizes mandatory fit and supported evidence	Rule and threshold quality must be calibrated and evaluated

keyword stuffing. Primary claims about real-world matching remain grounded in manually judged CV–JD pairs.

2.11 Research Gap

The literature provides strong methods for lexical retrieval, semantic retrieval, and ranking. However, many CV–JD systems still optimize a global similarity score. They may not explicitly distinguish mandatory from optional requirements, validate seniority constraints, or verify whether claimed skills are supported by experience evidence.

The gap addressed by this thesis is a practical multi-stage mechanism that:

- decomposes the JD into explicit requirements;
- retrieves evidence for each requirement from CV sections;
- distinguishes declared skills from evidenced skills;
- validates minimum experience and seniority constraints;
- reranks candidates based on requirement coverage and constraint satisfaction;
- explains each decision to the recruiter.

Chapter 3

System Design and Implementation

3.1 System Overview

The proposed system, named RAEMatch, is a recruiter-facing decision-support prototype. Its main workflow is shown in fig. 3.1. The system separates document ingestion, structured parsing, matching, reranking, and explanation into independent modules.



Figure 3.1: Proposed requirement-aware and evidence-aware matching architecture

3.2 Functional Requirements

The system shall:

1. upload one or more PDF CV files;
2. extract text directly or apply OCR fallback;

3. anonymize direct identifiers before ranking;
4. parse CV sections and JD requirements;
5. classify requirements as mandatory or optional;
6. extract experience years and seniority levels;
7. retrieve supporting evidence sentences;
8. assess requirement satisfaction;
9. rerank candidates using hard and soft constraints;
10. show top-k results and explanations;
11. store reviewer relevance labels and evaluation results.

3.2.1 Non-functional Requirements

The prototype is designed around five non-functional requirements: traceability, deterministic behavior, privacy protection, acceptable interactive response time, and modularity. Every ranking result retains the parsed requirements, requirement states, evidence sentences, constraint violations, and baseline tie-breaker. This trace allows a reviewer to reproduce why a candidate was positioned at a specific rank. Parsing, evidence retrieval, constraint validation, reranking, and explanation generation are implemented as separate modules so that each component can be tested independently.

3.3 PDF Ingestion and OCR Fallback

The main user-facing input is PDF. Direct extraction is attempted first because it is faster and generally more accurate for text-based PDFs. If the extracted text is empty or below a minimum quality threshold, the system renders the PDF pages and applies OCR.

Each upload returns a per-file status such as `pdf_text_extracted`, `pdf_ocr_extracted`, `unsupported_file_type`, `empty_text`, or `ocr_failed`. Unreadable files are excluded from ranking rather than silently assigned low relevance.

3.4 Structured CV and JD Parsing

The parser normalizes CV headings and extracts the following sections:

- Summary or Career Objective;
- Skills;
- Work Experience;
- Projects;
- Education;

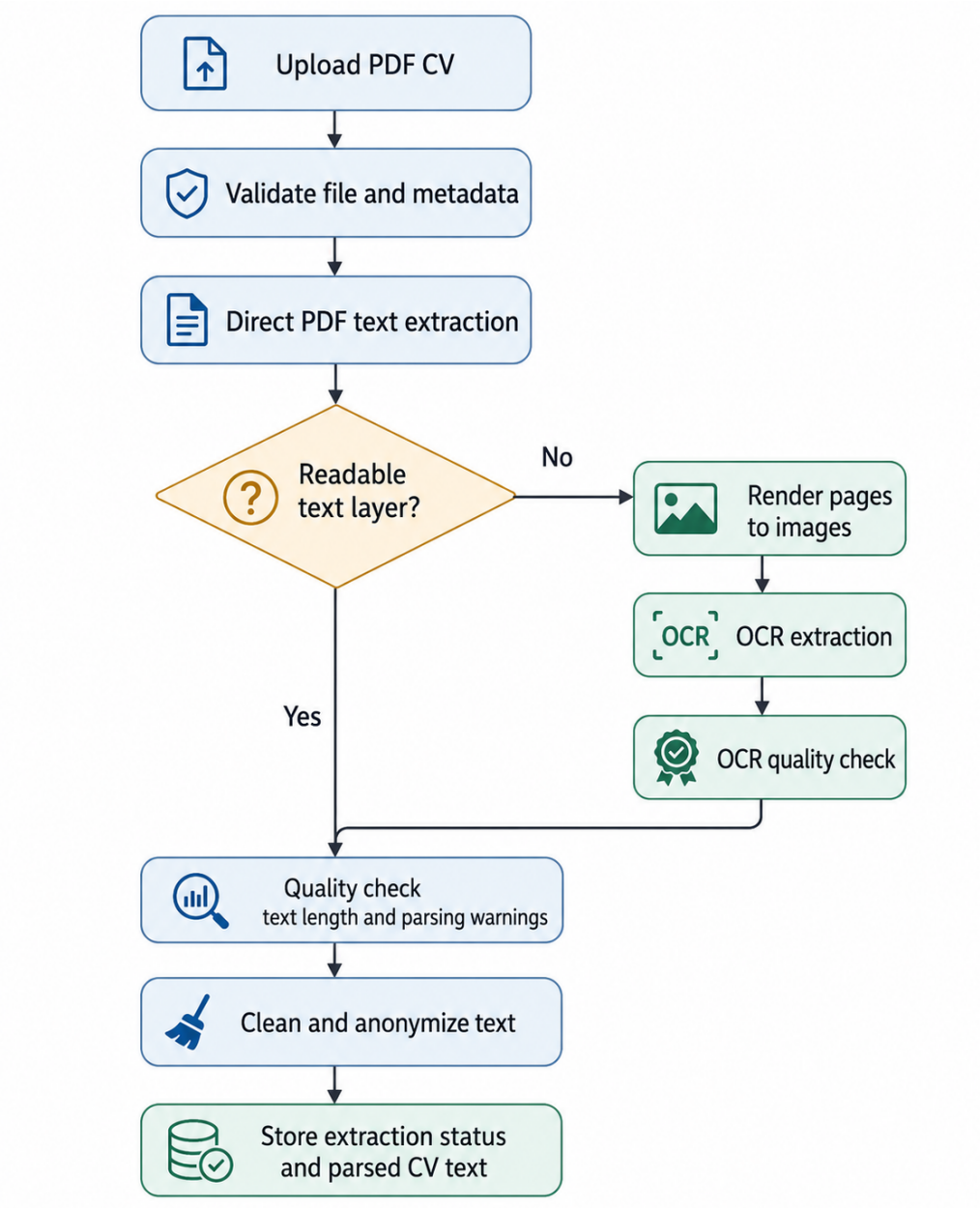


Figure 3.2: PDF ingestion and OCR fallback process

- Certifications.

The JD parser extracts job title, mandatory skills, optional skills, minimum years, required seniority, responsibilities, education, and certifications. A rule-based parser is used for the prototype because it is deterministic and auditable. Skill dictionaries and heading synonyms are configurable.

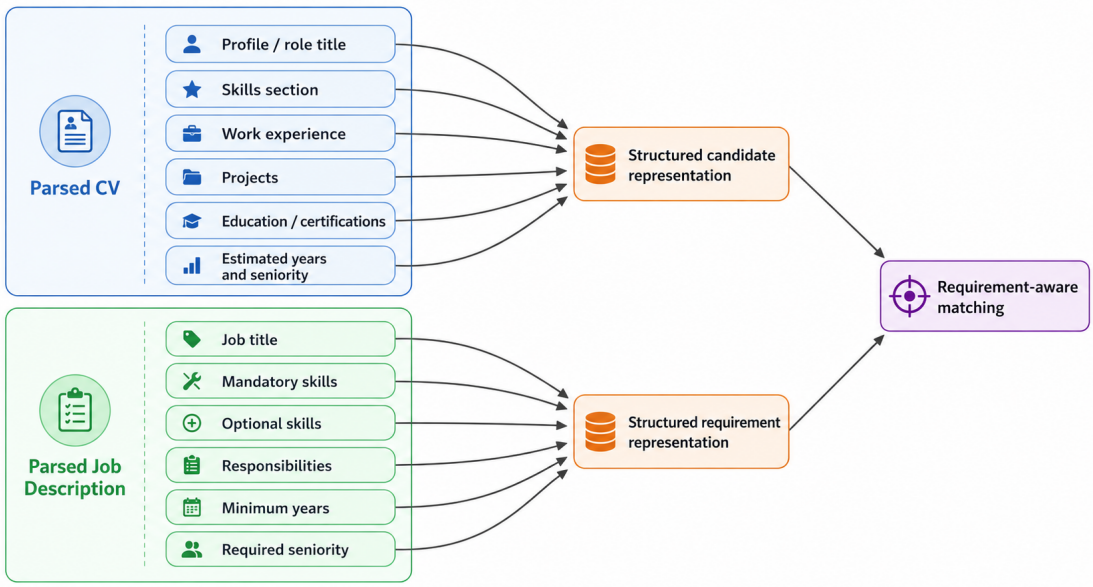


Figure 3.3: Structured parsing output for CV and JD documents

Table 3.1: Structured representation of a job description

Field	Example
Job title	Senior Java Developer
Mandatory skills	Java, Spring Boot, REST API, SQL
Optional skills	AWS, Kubernetes, Kafka
Minimum experience	5 years
Required seniority	Senior
Responsibilities	System design, production troubleshooting, mentoring
Education/certification	Bachelor degree; cloud certification preferred

3.5 Dataset Provenance and Source Tracking

Every CV record retains provenance metadata so that real and synthetic results can be analyzed separately. Real PDF records store the Kaggle source identifier, document type, OCR status, parsing warnings, and filtering outcome. Synthetic records store the generation

scenario and structured ground-truth profile used to create the CV. This separation prevents synthetic cases from being silently mixed with real-data conclusions.

Table 3.2: Core provenance and synthetic manifest fields

Field	Real PDF example	Synthetic example
CV identifier	`REAL_0042`	`SYN_0001`
Source type	`real_kaggle`	`synthetic`
Document type	`scanned_pdf`	`text_layer_pdf`
OCR status	`ocr_success`	`not_required`
Role / seniority	Parsed from CV	Controlled profile metadata
Experience years	Parsed or estimated	Controlled profile metadata
Evidenced skills	Extracted from Experience/Projects	Defined in generation manifest
Unsupported skills	Inferred from declarations without evidence	Controlled scenario metadata
Scenario type	`natural`	strong match, seniority mismatch, keyword stuffing, unsupported claim, or irrelevant role
Experiment split	development / evaluation	development / robustness / scalability

3.6 Synthetic CV Generation Protocol

Approximately 1,000 English IT CVs are generated from structured candidate profiles to increase corpus scale and create controlled edge cases. The final generator model, prompt version, and exact counts remain `Pending external input`. The protocol is fixed as follows:

1. define role, seniority, years of experience, responsibilities, evidenced skills, declared-only skills, education, and certifications;
2. generate CV text without exposing or copying the target evaluation JD;
3. convert the generated record into a consistent PDF format and retain a machine-readable manifest;
4. reject empty, duplicated, internally inconsistent, or excessively repetitive outputs;
5. sample records manually for language quality and profile consistency;
6. keep synthetic records separate from real-data effectiveness conclusions.

The role matrix includes backend, frontend, full-stack, QA automation, DevOps, data engineering, business analysis, project management, software architecture, and IT support.

The seniority matrix includes Intern, Fresher, Junior, Middle, Senior, Lead, and Architect. The final distribution is reported in Appendix A.2.

3.7 Requirement Decomposition

Each JD is converted into a set of atomic requirements:

$$R(q) = R_h(q) \cup R_s(q), \tag{3.1}$$

where R_h contains hard constraints and R_s contains soft preferences.

Examples of hard constraints are mandatory skills, minimum years, and required certification. Soft constraints include optional tools, preferred education, or desirable domain experience.

Each requirement is represented as:

$$r_j = (t_j, c_j, w_j), \tag{3.2}$$

where t_j is the requirement text, $c_j \in \{\text{hard}, \text{soft}\}$ is the constraint type, and w_j is its importance.

3.8 Candidate Evidence Retrieval

The system splits Work Experience and Project sections into evidence sentences. For each requirement r_j, it calculates semantic similarity against all evidence sentences and retrieves the most relevant sentence:

$$s_j^* = \arg\max_{s \in \mathcal{S}(d)} \cos(\mathbf{e}_{r_j}, \mathbf{e}_s). \tag{3.3}$$

The evidence score is:

$$E_j = \max_{s \in \mathcal{S}(d)} \cos(\mathbf{e}_{r_j}, \mathbf{e}_s). \tag{3.4}$$

A skill listed only in the Skills section is considered *declared*. It becomes *evidenced* only when a supporting sentence is found in Work Experience or Projects above the selected threshold.

3.9 Requirement Satisfaction Assessment

Each requirement receives one of four statuses:

Table 3.3: Requirement satisfaction states

Status	Definition	Example
Satisfied	Requirement is explicitly supported by strong evidence	Spring Boot used in a production project
Partially satisfied	Related evidence exists but is incomplete or indirect	General cloud deployment but no AWS-specific evidence
Unsupported	Skill is declared but no supporting experience evidence is found	Kubernetes appears only in Skills
Not satisfied	Requirement is missing or contradicted	JD requires 5 years; CV shows 1 year

A configurable mapping converts these statuses to ordered values for deterministic reranking:

$$\rho(r_j, d) = \begin{cases} 3, & \text{satisfied,} \\ 2, & \text{partially satisfied,} \\ 1, & \text{unsupported,} \\ 0, & \text{not satisfied.} \end{cases} \tag{3.5}$$

These values are not used as the research ground-truth labels. They are internal requirement states used by the mechanism.

3.9.1 Operational Decision Rules

To make the implementation reproducible, the prototype applies explicit decision rules. Exact skill evidence in Work Experience or Projects is classified as satisfied. Semantically related evidence above the high threshold τ_h is also satisfied, while evidence between the lower threshold τ_l and τ_h is partially satisfied. A skill found only in the declared Skills section is unsupported. Requirements with no declaration or evidence are not satisfied. The initial thresholds are configuration parameters rather than universal constants and are calibrated on a development subset before final evaluation.

The final values of τ_l and τ_h are recorded with the experiment configuration. This prevents threshold selection from being hidden and allows sensitivity analysis after evaluation evidence is available.

Table 3.4: Operational requirement assessment rules

Status	Operational rule	Prototype configuration
Satisfied	Exact evidence or semantic evidence at or above the high threshold	$E_j \geq \tau_h$
Partially satisfied	Related but incomplete evidence between the two thresholds	$\tau_l \leq E_j < \tau_h$
Unsupported	Requirement term declared in Skills but no evidence in Experience or Projects	Declaration only
Not satisfied	No valid declaration/evidence, or an explicit constraint contradiction	$E_j < \tau_l$

3.10 Experience and Seniority Validation

Years of experience are extracted from explicit phrases and, where possible, employment date ranges. Seniority is mapped to the ordered scale:

$$\text{Intern} < \text{Fresher} < \text{Junior} < \text{Middle} < \text{Senior} < \text{Lead} < \text{Architect}. \tag{3.6}$$

Experience-year alignment is defined as:

$$Y_{\text{align}}(d, q) = \min\left(\frac{Y_d}{Y_q}, 1\right), \tag{3.7}$$

where Y_d is the candidate's estimated experience and Y_q is the JD minimum. If no minimum is specified, the component is treated as not applicable.

Seniority-level alignment is defined using the ordinal distance $\Delta_L = L_q - L_d$:

$$L_{\text{align}}(d, q) = \begin{cases} 1.0, & \Delta_L \leq 0, \\ 0.5, & \Delta_L = 1, \\ 0.0, & \Delta_L \geq 2. \end{cases} \tag{3.8}$$

The combined alignment is:

$$S_{\text{align}}(d, q) = \frac{Y_{\text{align}}(d, q) + L_{\text{align}}(d, q)}{2}. \tag{3.9}$$

The final implementation records missing or ambiguous dates rather than silently assuming zero experience.

3.11 Constraint-based Reranking

The proposed method does not calculate one weighted final similarity score. Candidates are ordered by a lexicographic ranking key:

$$K(d, q) = \left(H_{\text{pass}}, C_h, E_h, S_{\text{align}}, C_s, E_s, B\right), \tag{3.10}$$

where the components are defined in Table 3.5.

Table 3.5: Definition of constraint-based reranking components

Component	Operational definition
H_{pass}	1 when no hard requirement is not satisfied; otherwise 0.
C_h	Sum of normalized satisfaction values for hard requirements divided by the number of hard requirements.
E_h	Number of hard requirements with valid Experience/Project evidence divided by the number of hard requirements.
S_{align}	Mean of experience-year and seniority-level alignment.
C_s	Sum of normalized satisfaction values for soft requirements divided by the number of soft requirements.
E_s	Number of soft requirements with valid supporting evidence divided by the number of soft requirements.
B	SBERT full-document similarity used only as the final tie-breaker.

The requirement-state mapping $\rho \in \{0, 1, 2, 3\}$ is normalized by dividing by three before coverage calculation. Empty hard- or soft-requirement groups are handled by neutral defaults documented in the experiment configuration. Candidates are sorted lexicographically in descending order. Consequently, a candidate who passes all hard constraints is prioritized over one who fails a hard constraint, even when the latter has higher broad semantic similarity.

3.11.1 Reranking Procedure

Listing 3.1: Requirement-aware and evidence-aware reranking procedure

```
Input: job description q, CV corpus D, top-k value k
Output: ranked candidates with explanations

1. Parse q into hard requirements Rh and soft requirements Rs.
2. For each CV d in D:
3.     Parse CV sections and extraction-quality metadata.
4.     Extract candidate experience years and seniority.
5.     For each requirement r in Rh union Rs:
```

```
6.          Retrieve the highest-similarity evidence sentence.
7.          Assign satisfied, partial, unsupported, or not-
   satisfied status.
8.      Validate hard requirements and calculate Hpass.
9.      Calculate Ch, Eh, Salign, Cs, Es, and baseline score B.
10.     Build ranking key K(d,q).
11. Sort candidates lexicographically by K(d,q), descending.
12. Return the top-k candidates and requirement-level
   explanations.
```

3.12 Baseline Methods

The following baselines are retained for comparison:

1. **TF–IDF full text**: lexical cosine similarity.
2. **SBERT full text**: full-document semantic similarity.
3. **Section-level SBERT**: separate similarity for Skills, Experience, Education, and overall text.
4. **Requirement-aware**: requirement decomposition and coverage without evidence verification.
5. **Full proposed model**: requirement-aware, evidence-aware, seniority-aware, constraint-based reranking.

3.13 Explanation Generation

For each candidate, the explanation page displays:

- hard constraints passed and violated;
- satisfied, partially satisfied, unsupported, and missing requirements;
- required and candidate experience years;
- required and candidate seniority;
- supporting evidence sentences;
- OCR or parsing warnings;
- reviewer relevance label and note.

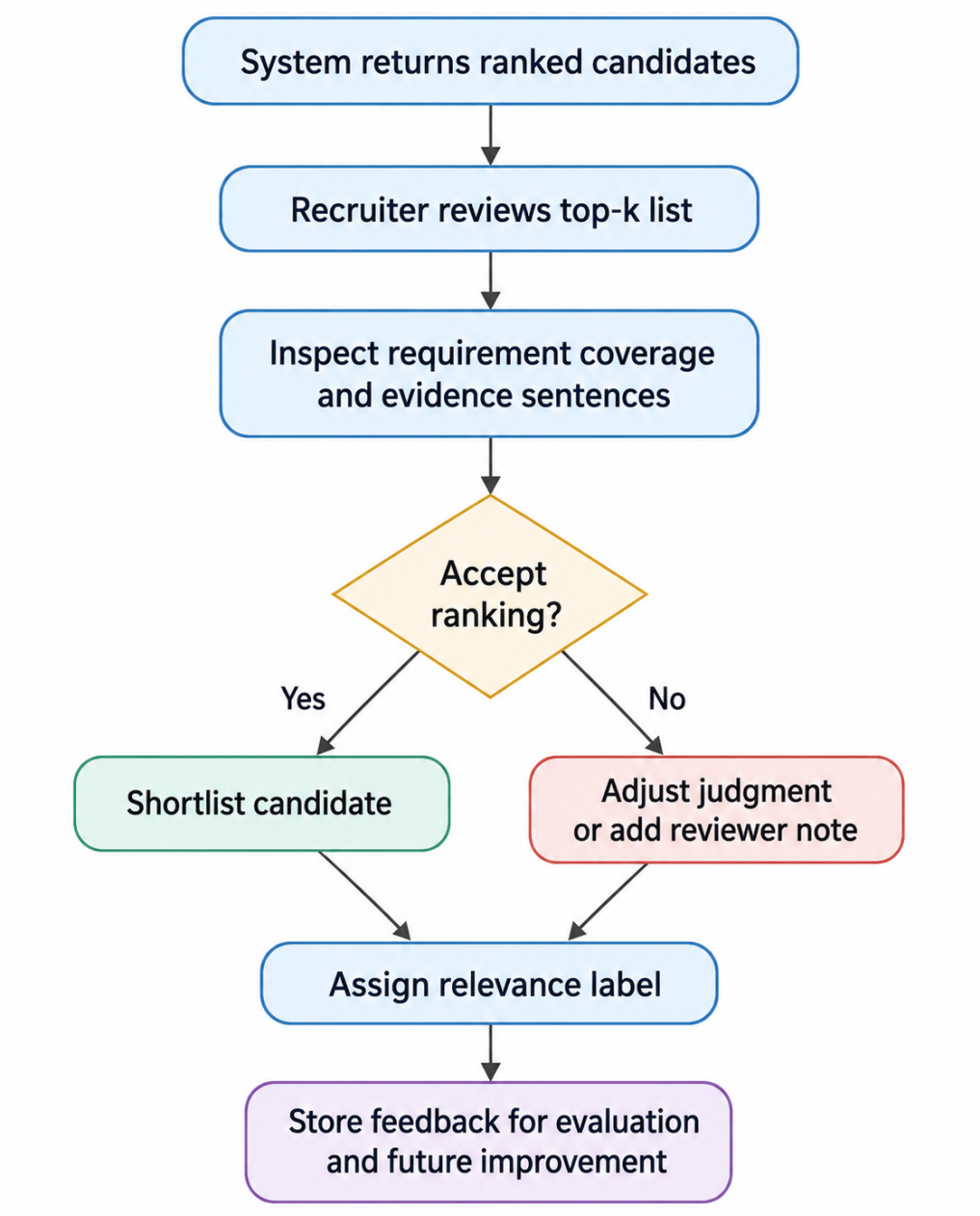


Figure 3.4: Human-in-the-loop recruiter review workflow

3.14 Software Architecture

The backend is implemented with Python and FastAPI. PyPDF handles text-based PDF extraction, while pdf2image and Tesseract provide OCR fallback. Scikit-learn implements TF–IDF. Sentence-transformers provides SBERT embeddings. SQLite is sufficient for the prototype; PostgreSQL and FAISS are deployment options for larger corpora. The frontend is implemented in ReactJS.

Table 3.6: Implementation and experiment environment

Item	Recorded configuration
Backend runtime	Python and FastAPI versions from the final build
Frontend runtime	Node.js and React versions from the final build
Embedding library/model	sentence-transformers version and exact SBERT checkpoint
PDF/OCR stack	PyPDF, pdf2image, Tesseract, and Poppler versions
Storage	SQLite prototype database
Execution hardware	Processor, memory, and operating system of the evaluation machine
Randomness	Random seed for sampling and any non-deterministic operation

These fields are separated from the algorithm description so the final experiment can be reproduced without changing the proposed mechanism.

3.15 API Design

3.16 Security and Human-in-the-loop Controls

Direct identifiers such as names, emails, phone numbers, and exact addresses are masked before ranking. File types and sizes are validated. Raw CV content is not sent to external APIs in the local prototype. The system does not automatically reject candidates. Recruiters remain responsible for final screening and hiring decisions.

Table 3.7: Main software modules

Module	Responsibility
PDF ingestion	Validation, direct extraction, OCR fallback, quality flags
Preprocessing	Cleaning, anonymization, duplicate detection
CV parser	Section extraction, sentence segmentation, skills, dates
JD parser	Mandatory/optional requirements, years, seniority, responsibilities
Evidence retriever	Requirement-to-sentence semantic matching
Constraint engine	Hard/soft validation and seniority checks
Reranking engine	Lexicographic requirement-based ordering
Explanation service	Requirement states, evidence, gaps, warnings
Dataset provenance service	Source type, OCR status, scenario metadata, and experiment split
Synthetic manifest loader	Controlled profile and generation-scenario import
Evaluation service	Labels, ranking metrics, ablation, robustness, latency, and usability tests

Table 3.8: Main API endpoints

Endpoint	Method	Purpose
`/api/cvs/upload-pdfs`	POST	Upload PDF CVs, extract text, apply OCR fallback, and index valid records
`/api/cvs`	GET	List indexed CV profiles and parsing status
`/api/jds`	POST	Create and parse a JD
`/api/jds/{id}/requirements`	GET	Retrieve decomposed JD requirements
`/api/jds/{id}/rank`	POST	Run a baseline or the proposed reranking mechanism
`/api/explanations/{jd}/{cv}`	GET	Retrieve requirement-level explanation and evidence
`/api/labels`	POST	Save reviewer relevance labels
`/api/experiments/metrics`	GET	Return ranking evaluation metrics

Chapter 4

Evaluation and Discussion

4.1 Evaluation Goals

The evaluation is designed to answer whether the proposed mechanism improves CV–JD matching rather than merely producing a score. It measures:

- ranking effectiveness;
- reduction of seniority-mismatch false positives;
- resistance to keyword stuffing and unsupported skill claims;
- PDF extraction and OCR reliability;
- explanation usefulness and error characteristics.

4.2 Dataset and Test Corpus

The evaluation corpus is organized into distinct subsets so that real-world effectiveness, controlled robustness, and system scalability are not conflated. Exact counts, dataset URLs, licenses, and filtering outcomes remain `Pending external input` until the final corpus audit is completed.

4.2.1 Real Scanned PDF Corpus

The real corpus consists of several hundred scanned resume PDFs obtained from a public Kaggle dataset. Because the records are image-based, OCR is required before parsing. The final report records the exact dataset title, Kaggle URL, license, original file count, English-language count, IT-domain count, duplicate-removal count, OCR success and failure counts, and final usable corpus size.

The real corpus is used for:

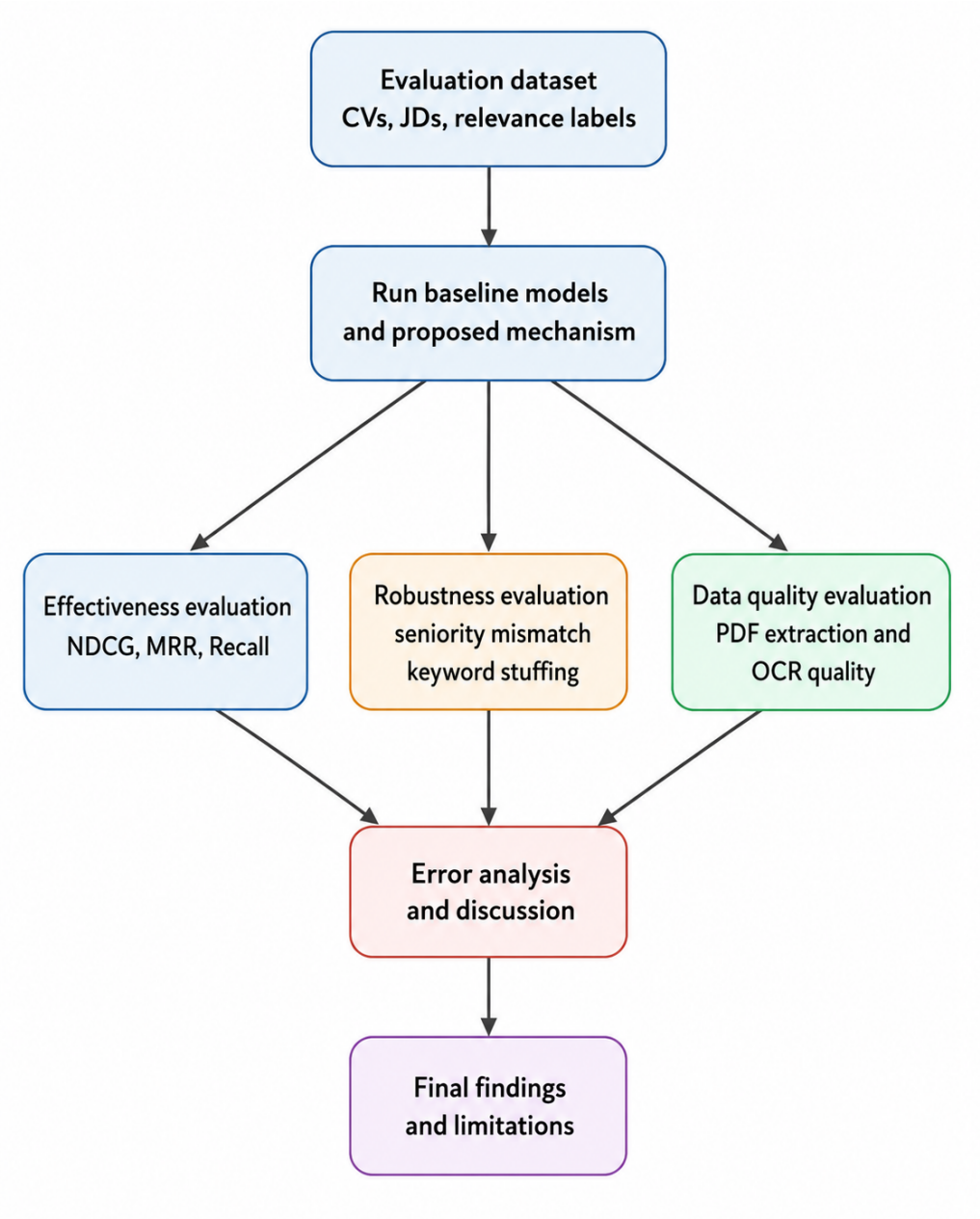


Figure 4.1: Evaluation workflow for effectiveness, robustness, and data quality

- OCR and document-ingestion evaluation;
- section, skill, experience, and seniority parsing analysis;
- human-labeled CV–JD ranking evaluation;
- real-world case studies and error analysis.

4.2.2 Controlled Synthetic CV Corpus

Approximately 1,000 AI-generated English IT CVs are used to expand corpus size and create controlled combinations of role, seniority, years, responsibilities, evidenced skills, unsupported claims, and irrelevant skills. Synthetic CVs are used for robustness and scalability analysis, not as the sole evidence of practical matching effectiveness.

Each synthetic record includes a manifest containing role, seniority, years of experience, declared skills, evidence-supported skills, unsupported skills, responsibility profile, scenario type, generation prompt version, and quality-control status. The final model name, prompt template, role distribution, seniority distribution, and accepted/rejected counts remain `Pending external input`.

4.2.3 Combined Retrieval Corpus

The combined retrieval corpus contains all valid real CVs and accepted synthetic CVs. It is used to measure indexing time, ranking latency, top-k response time, and behavior at a corpus size of approximately 1,000 plus the usable real scanned records. Effectiveness results are still reported separately for real and synthetic subsets.

4.2.4 Human-labeled Evaluation Pool

The primary ranking evaluation uses a manually judged pool of CV–JD pairs. The target design contains five representative IT JDs and 30–50 candidate CVs per JD. The majority of judged pairs should come from the real Kaggle corpus, while synthetic cases supplement controlled edge conditions. The recommended initial target is 150–250 unique pairs, with the exact number marked `Pending reviewer availability`.

Candidate pooling combines top results from TF–IDF, SBERT, section-level SBERT, requirement-aware matching, the full proposed model, and random sampling. Duplicate candidates are removed before judgment. This pooling process reduces bias toward any single model.

4.2.5 Controlled Robustness Sets

Separate controlled sets are constructed for:

- seniority mismatch: same or similar technology stack with different experience and ownership levels;
- keyword stuffing: skills added to the Skills section without supporting evidence;
- supported skill addition: skills added together with relevant Experience or Project evidence;
- missing mandatory requirement: strong broad similarity but one or more hard requirements absent;
- role confusion: related IT roles with overlapping terminology but different responsibilities.

Table 4.1: Dataset composition for the final experiment

Subset	Source	Final size	Primary purpose
Real scanned PDF corpus	Kaggle	Pending	OCR, parsing, real-data ranking, and error analysis
Synthetic CV corpus	AI-generated	Approximately 1,000	Controlled robustness and scalability
Combined retrieval corpus	Mixed	Pending	Indexing and ranking latency
Human-labeled evaluation pool	Primarily real, selected synthetic	Pending	NDCG, MAP, MRR, Recall, and per-JD analysis
Seniority robustness set	Controlled synthetic and selected real	Pending	Hard-constraint violation analysis
Keyword-stuffing set	Controlled paired variants	Pending	Unsupported-claim and rank-change analysis

4.3 Relevance Labeling

Reviewer labels use a 0–3 graded scale. Human judgments are the primary ground truth for ranking effectiveness. Synthetic manifest labels are used only to verify controlled scenarios and do not replace reviewer relevance judgments:

Reviewers evaluate mandatory skills, optional skills, experience relevance, minimum years, seniority, responsibility coverage, and evidence strength. The following rubric standardizes judgment without replacing reviewer expertise.

The percentages guide reviewer attention; they are not used as the proposed ranking formula. If two reviewers complete the judgments, weighted Cohen's kappa is reported [16]. If only one reviewer is available, the final report states this limitation and preserves reviewer notes

Table 4.2: Dataset documentation fields for the final experiment

Field	Final value
Public dataset name, URL, and license	Pending external input
Original scanned PDF count	Pending
English IT CVs after filtering	Pending
OCR success / failure counts	Pending
Duplicates and invalid records removed	Pending
Synthetic generator and prompt version	Pending external input
Accepted synthetic CVs	Pending
Evaluation JDs and roles	Pending
Unique labeled CV–JD pairs	Pending reviewer input
Development/evaluation split	Pending final protocol

Table 4.3: Manual relevance scale

Label	Meaning	Reviewer interpretation
0	Not relevant	Fails core role requirements or belongs to another role
1	Weakly relevant	Some overlap but major mandatory, experience, or seniority gaps
2	Relevant	Satisfies several important requirements with acceptable gaps
3	Highly relevant	Strong mandatory, experience, seniority, responsibility, and evidence match

Table 4.4: Reviewer labeling rubric

Criterion	Guidance weight	Review guidance
Mandatory skill fit	30%	Confirm whether core technologies or equivalent competencies are present and evidenced
Relevant experience	25%	Assess role and responsibility relevance, not only job title
Minimum years	15%	Compare explicit or date-derived experience with the JD requirement
Seniority alignment	15%	Consider ownership, design, production support, and mentoring evidence
Responsibility evidence	15%	Verify that important JD responsibilities have supporting CV sentences

for auditability.

4.3.1 Reviewer Agreement and Explanation Usability

If two reviewers are available, weighted Cohen's kappa is reported [16] for the graded labels. If only one reviewer is available, the report states the reviewer profile, preserves comments, and treats single-reviewer judgment as a limitation.

To answer RQ5, a small pilot usability assessment is conducted with `Pending` recruiters, technical leads, or experienced interviewers. Participants review selected ranked candidates and rate explanation clarity, usefulness, trust, and ability to identify candidate gaps on a five-point Likert scale. If the pilot cannot be completed, RQ5 and the corresponding conclusions are explicitly limited to the design and content of the explanation output rather than demonstrated user benefit.

Table 4.5: Explanation usability reporting template

Item	Final result
Number and profile of participants	Pending
Explanation clarity (1–5)	Pending
Usefulness for identifying gaps (1–5)	Pending
Trust in requirement-level evidence (1–5)	Pending
Perceived screening support (1–5)	Pending
Qualitative comments	Pending

4.4 Experimental Models

The model comparison is performed in three experiment groups:

1. **Real-data effectiveness**: human-labeled pairs, reporting ranking metrics and per-JD analysis;
2. **Controlled robustness**: seniority mismatch, unsupported claims, and keyword-stuffing cases;
3. **Combined-corpus performance**: indexing, latency, and response-time measurements.

Table 4.6: Experimental model comparison

ID	Model	Purpose
M1	TF–IDF full text	Lexical baseline
M2	SBERT full text	Semantic baseline
M3	Section-level SBERT	Section-aware baseline
M4	Requirement-aware matching	Tests JD decomposition and requirement coverage
M5	Requirement + seniority constraints	Measures reduction of experience and seniority mismatch
M6	Full proposed model	Requirement-aware, evidence-aware, constraint-based reranking

4.5 Ranking Metrics

The main metrics are:

- **NDCG@5 and NDCG@10**: reward highly relevant candidates near the top;
- **MAP**: evaluates precision across relevant candidates;
- **MRR**: measures how early the first relevant candidate appears;
- **Recall@10 and Recall@20**: measure shortlist coverage;
- **Constraint violation rate@k**: proportion of top-k candidates that violate hard constraints;
- **Unsupported claim rate@k**: proportion of matched skills without supporting evidence.

The additional rates directly test whether the proposed mechanism solves the practical false-positive problem.

4.6 Ablation Study

The ablation study adds one mechanism at a time:

$$\text{SBERT} \rightarrow \text{Section matching} \rightarrow \text{Requirements} \rightarrow \text{Seniority constraints} \rightarrow \text{Evidence verification.} \tag{4.1}$$

This design shows whether each component contributes to ranking improvement. It also prevents the discussion from relying only on an opaque final model. The ablation study is run on the human-labeled evaluation pool, while controlled synthetic cases are reported separately.

4.6.1 Threshold Sensitivity

The lower and higher evidence thresholds τ_l and τ_h are calibrated on a development subset and frozen before final evaluation. A small sensitivity analysis reports whether moderate threshold changes alter NDCG@10, hard-constraint violation rate, or unsupported-claim rate. Final threshold values and the explored range remain `Pending final experiment`.

4.7 Seniority Mismatch Robustness Test

The test set includes JDs with explicit minimum years and seniority levels. Candidate CVs are grouped as Intern/Fresher, Junior, Middle, Senior, Lead, and Architect. The main measurement is whether underqualified candidates appear in the top-k list.

Example test:

> JD: Senior Java Developer, minimum 5 years, system design and mentoring.
> CV A: Fresher with Java, Spring Boot, SQL, Docker.
> CV B: Senior candidate with 6 years, microservices, production troubleshooting, and mentoring.

Similarity-only methods may rank CV A highly because of skill overlap. The proposed mechanism should rank CV B above CV A because the hard constraints and evidence are stronger.

4.8 Keyword-stuffing Robustness Test

A controlled adversarial test is created by adding unrelated or unsupported skills to a CV Skills section without adding project or experience evidence. The rank change is measured before and after manipulation.

Table 4.7: Keyword-stuffing test design

Version	Modification	Expected behavior
Original CV	Skills and evidence remain consistent	Stable ranking
Stuffed CV	Add AWS, Kubernetes, Kafka, Spark, React, TensorFlow to Skills only	Similarity baseline may increase; evidence-aware rank should not materially increase
Supported CV	Add the same skills with project evidence	Ranking may increase if requirements are relevant

4.9 PDF and OCR Evaluation

The ingestion evaluation reports:

- direct extraction success rate;
- OCR fallback success rate;
- extracted character count;
- section detection completeness;
- processing time per CV;
- failure reasons.

OCR reliability is evaluated only on the real scanned Kaggle subset. Synthetic text-layer PDFs are excluded from the OCR success rate and are used instead for controlled ranking and scalability tests. OCR is not the primary research contribution, but extraction quality is documented because it directly affects downstream evidence retrieval and ranking.

4.10 Result Tables

The table structures below are fixed before experimentation to reduce selective reporting. Quantitative cells remain `Pending final experiment` and must be populated only from reproducible runs.

4.11 Case-study Analysis

Two controlled case studies connect metric results to the original problem statement. The first compares a keyword-overlapping junior CV with a senior CV for a Senior Java Developer JD. The second compares an original CV with a keyword-stuffed version in which skills are added without work or project evidence.

Table 4.8: Real-data ranking effectiveness results

Model	NDCG@5	NDCG@10	MAP	MRR	Recall@10	Recall@20
TF–IDF	Pending	Pending	Pending	Pending	Pending	Pending
SBERT full text	Pending	Pending	Pending	Pending	Pending	Pending
Section-level SBERT	Pending	Pending	Pending	Pending	Pending	Pending
Requirement-aware	Pending	Pending	Pending	Pending	Pending	Pending
Requirement + seniority	Pending	Pending	Pending	Pending	Pending	Pending
Full proposed model	Pending	Pending	Pending	Pending	Pending	Pending

Table 4.9: Controlled robustness results

Model	Hard-constraint violation@10	Unsupported-claim rate@10	Mean rank change after stuffing
SBERT full text	Pending	Pending	Pending
Section-level SBERT	Pending	Pending	Pending
Requirement + seniority	Pending	Pending	Pending
Full proposed model	Pending	Pending	Pending

Table 4.10: Real scanned PDF OCR and parsing results

Metric	Final result
Total scanned PDFs	Pending
OCR success rate	Pending
OCR failure rate	Pending
Average OCR time per page	Pending
Skills-section detection rate	Pending
Experience-section detection rate	Pending
Experience-year correctness on inspected sample	Pending
Seniority correctness on inspected sample	Pending

Table 4.11: Combined-corpus operational performance

Operation	Corpus / unit	Final result
Text-layer extraction	per CV	Pending
OCR extraction	per page	Pending
Parsing and indexing	per CV	Pending
Initial index build	combined corpus	Pending
Ranking	per JD, combined corpus	Pending
Explanation generation	per candidate	Pending
Top-k API response	combined corpus	Pending

Table 4.12: Seniority mismatch case-study template

Candidate	Baseline rank	Proposed rank	Requirement-level explanation
Junior/fresher Java CV	Pending	Pending	Years gap, seniority mismatch, missing design/mentoring evidence
Senior Java CV	Pending	Pending	Years satisfied, seniority aligned, production/design evidence

Table 4.13: Keyword-stuffing case-study template

CV version	Baseline rank	Proposed rank	Interpretation
Original CV	Pending	Pending	Consistent declaration and evidence
Stuffed CV	Pending	Pending	Added skills without Experience/Project evidence
Evidence-supported CV	Pending	Pending	Added relevant skills together with supporting evidence

4.12 Operational Performance

Operational measurements are collected on the documented evaluation machine after one warm-up run. The final report states corpus size, hardware, software versions, and whether embedding vectors were precomputed. Results are reported in Table 4.11. The purpose is to establish prototype feasibility, not production-scale throughput.

4.13 Error Analysis

Error analysis inspects both false positives and false negatives and records the responsible pipeline stage.

4.14 Discussion

The final discussion is organized by research question rather than by model name. Real-data ranking metrics answer whether requirement-aware matching improves ordering on manually judged CV–JD pairs. Hard-constraint violation rate measures whether experience and seniority mismatches are reduced. Unsupported-claim rate and paired keyword-stuffing

Table 4.14: Error analysis categories

Error type	Possible cause	Example
False positive	Seniority mismatch	Fresher Java CV ranked for a Senior 5+ years JD
False positive	Unsupported skills	Kubernetes listed without project or experience evidence
False positive	Broad semantic similarity	Project manager CV matches software-delivery terminology
False negative	Implicit evidence	Candidate performed system design but used different wording
False negative	Parsing/OCR failure	Work Experience sentence lost during extraction
False negative	Requirement parser error	Mandatory skill incorrectly classified as optional

tests measure whether evidence verification limits ranking inflation. OCR and runtime results establish whether the software pipeline is sufficiently reliable for the experiment. The explanation-usability pilot addresses whether the output is understandable and useful to reviewers.

Results from synthetic controlled cases are interpreted as evidence of mechanism behavior under known conditions, not as equivalent to operational recruiter performance. Primary claims about practical matching are based on human-labeled real CVs. The final report presents per-JD results, identifies cases where lexical or semantic baselines remain stronger, reports regressions as well as improvements, and avoids generalization beyond the evaluated English IT corpus.

Until quantitative evidence is inserted, this chapter deliberately makes no claim that the proposed model outperforms the baselines.

4.15 Threats to Validity

4.15.1 Internal Validity

Errors in PDF extraction, OCR, requirement parsing, date extraction, sentence segmentation, or threshold selection may affect measured performance. Mitigations include extraction-quality logs, development/evaluation separation, frozen configurations, controlled test cases, and manual inspection of a sample. Synthetic prompt leakage is mitigated by generating CVs from structured profiles independently of target evaluation JDs.

4.15.2 Construct Validity

Relevance labels are subjective and may vary by reviewer. The fixed rubric, reviewer comments, and weighted Cohen's kappa when two reviewers are available reduce ambiguity. Internal requirement states are not treated as ground-truth relevance labels. Explanation usefulness is measured separately from ranking effectiveness.

4.15.3 External Validity

The evaluation focuses on English IT recruitment. Public Kaggle CVs may not represent company-specific documents, and AI-generated CVs may contain cleaner language or generator-specific patterns. Synthetic results are therefore reported separately and are not used to claim production readiness.

4.15.4 Conclusion Validity

A limited number of JDs and judged pairs may produce unstable metric differences. The final report includes per-JD results, bootstrap confidence intervals where feasible, and sensitivity analysis for evidence thresholds. Negative or inconclusive outcomes are reported rather than hidden.

Chapter 5

Conclusion and Future Work

5.1 Conclusion

This thesis reframes recruitment screening as more than a whole-document text-similarity problem. Similarity-only ranking can return false positives when CVs share keywords with a JD but fail mandatory experience, seniority, responsibility, or evidence requirements. To address this limitation, the proposed RAEMatch system decomposes each JD into atomic requirements, retrieves supporting evidence from CV Work Experience and Projects, validates hard and soft constraints, and reranks candidates using a deterministic lexicographic key.

The system also implements a PDF-first ingestion workflow with direct extraction and OCR fallback. The evaluation design separates three forms of evidence: real scanned PDFs for OCR, parsing, and human-labeled ranking; controlled synthetic CVs for robustness and scalability; and a small reviewer study for explanation usability. It preserves human control by returning a top-k shortlist with requirement-level explanations rather than automatic hiring decisions.

The final conclusion will be updated with measured changes in NDCG, MRR, Recall, hard-constraint violation rate, unsupported-claim rate, OCR success, latency, and explanation-usability ratings. These quantitative conclusions remain `Pending final experiment`.

5.2 Main Contributions

The thesis design and implemented prototype contribute:

1. a five-chapter formulation of English IT CV–JD ranking;
2. a PDF ingestion and OCR fallback pipeline for real scanned CVs;
3. structured CV, JD, and dataset-provenance representations;
4. requirement decomposition into hard and soft constraints;

5. evidence retrieval from Experience and Project sentences;
6. explicit experience-year and seniority validation;
7. deterministic lexicographic constraint-based reranking;
8. recruiter-facing requirement-level explanations;
9. a transparent synthetic CV generation and manifest protocol;
10. an evaluation design covering real-data effectiveness, controlled robustness, OCR reliability, operational performance, and explanation usability.

5.3 Limitations

The main limitations are:

- public Kaggle data may not fully represent organizational recruitment;
- AI-generated CVs may contain stylistic artifacts, overly clean language, or prompt-specific patterns;
- synthetic metadata does not replace human relevance judgment;
- rule-based JD and CV parsing may fail on unusual formats or implicit requirements;
- experience-year extraction may be inaccurate when dates are incomplete or overlapping;
- OCR may introduce noisy text for low-resolution and multi-column scanned PDFs;
- semantic evidence thresholds require calibration and may vary across roles;
- evidence in a CV does not prove a claim is true; the system detects only internal support or absence of support;
- reviewer labels and usability participants may be limited;
- the prototype does not perform external employment verification or final hiring decisions.

5.4 Future Work

Future work may:

- replace deterministic reranking with LambdaMART or another learning-to-rank model [11];
- use a cross-encoder for requirement-to-evidence verification [12];
- build an IT skill ontology for related, equivalent, and prerequisite skills;
- improve JD requirement extraction using supervised or LLM-assisted structured parsing;
- compare multiple synthetic-data generators and adversarial generation strategies;

- validate the system on private organizational CVs under approved privacy controls;
- improve OCR for complex multi-column and low-resolution CVs;
- extend the system to Vietnamese and multilingual recruitment;
- conduct a larger user study with recruiters and technical interviewers;
- integrate fairness auditing, role-based access control, audit logging, and production monitoring.

Appendix A

Evaluation Artifacts

A.1 Final Dataset Record

This appendix records the exact public dataset title, Kaggle URL, license, original scanned PDF count, filtering script version, OCR success and failure counts, duplicate-removal results, final real corpus size, JD list, and labeled-pair count. All values are `Pending external input or final corpus audit`.

A.2 Synthetic Generation Protocol

The final appendix includes:

- generator model and version: `Pending`;
- prompt template and prompt version: `Pending`;
- role and seniority distributions: `Pending`;
- accepted and rejected generations: `Pending`;
- quality checks and duplicate-detection rules;
- a sample manifest containing role, seniority, years, evidenced skills, unsupported skills, responsibilities, and scenario type;
- the rule that target evaluation JDs are not copied into generation prompts.

A.3 Labeling Instructions

Reviewers receive the rubric in Table 4.4, anonymized candidate identifiers, the target JD, and a comment field. They do not use names, demographic attributes, photographs, or exact addresses. The final appendix includes one example for each relevance level and records reviewer profile and agreement statistics.

A.4 Experiment Configuration

The final configuration records the exact SBERT checkpoint, τ_l, τ_h, skill dictionary version, parsing-rule version, software versions, random seed, development/evaluation split, hardware, metric implementation, and commands used to reproduce ranking and metric calculation. Values remain `Pending final experiment`.

A.5 Prototype Evidence

Final application screenshots are inserted after the experiment build is frozen:

1. PDF upload and per-file OCR/extraction status;
2. parsed JD and requirement decomposition;
3. top-k ranking results;
4. candidate requirement-level explanation;
5. reviewer labeling interface or exported judgment form.

A.6 Pending Completion Checklist

Table A.1: Items requiring external input or experiment execution

Item	Status
Kaggle dataset name, URL, license, and audited counts	Pending
Synthetic generator, prompts, distributions, and quality-control results	Pending
Final evidence thresholds and sensitivity analysis	Pending
Human-labeled CV–JD pairs and reviewer agreement	Pending
Ranking, robustness, OCR, and latency results	Pending
Case-study ranks and explanations	Pending
Explanation-usability participants and ratings	Pending
Application screenshots and final environment versions	Pending
Abstract and conclusion quantitative result sentences	Pending

References

[1] A. H. Alderham and E. S. Jaha, "Comparative Semantic Resume Analysis for Improving Candidate-Career Matching," *2022 14th International Conference on Computational Intelligence and Communication Networks*, pp. 313–321, 2022.

[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," *Proceedings of NAACL-HLT*, 2019.

[3] K. Järvelin and J. Kekäläinen, "Cumulated Gain-based Evaluation of IR Techniques," *ACM Transactions on Information Systems*, vol. 20, no. 4, pp. 422–446, 2002.

[4] A. Jacovi and Y. Goldberg, "Towards Faithfully Interpretable NLP Systems: How Should We Define and Evaluate Faithfulness?" *Proceedings of ACL*, 2020.

[5] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[6] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," *Proceedings of EMNLP-IJCNLP*, 2019.

[7] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why Should I Trust You? Explaining the Predictions of Any Classifier," *Proceedings of KDD*, 2016.

[8] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[9] G. Salton and C. Buckley, "Term-weighting Approaches in Automatic Text Retrieval," *Information Processing and Management*, vol. 24, no. 5, pp. 513–523, 1988.

[10] A. Vaswani et al., "Attention Is All You Need," *Advances in Neural Information Processing Systems*, 2017.

[11] C. J. C. Burges, "From RankNet to LambdaRank to LambdaMART: An Overview," Microsoft Research Technical Report MSR-TR-2010-82, 2010.

[12] R. Nogueira and K. Cho, "Passage Re-ranking with BERT," *arXiv preprint arXiv:1901.04085*, 2019.

[13] L. Long et al., "On LLMs-Driven Synthetic Data Generation, Curation, and Evaluation: A Survey," *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

[14] Z. Tan et al., "Large Language Models for Data Annotation and Synthesis: A Survey," *Proceedings of EMNLP 2024*, 2024.

[15] R. Smith, "An Overview of the Tesseract OCR Engine," *Proceedings of the Ninth International Conference on Document Analysis and Recognition*, pp. 629–633, 2007.

[16] J. Cohen, "Weighted Kappa: Nominal Scale Agreement with Provision for Scaled Disagreement or Partial Credit," *Psychological Bulletin*, vol. 70, no. 4, pp. 213–220, 1968.